Design and Analysis of Algorithms

Chapter 1: Foundations and Asymptotic Complexity

1.1 Asymptotic Analysis and Big-O Notation

Asymptotic analysis evaluates algorithm performance as input size grows toward infinity.
Big-O notation represents an asymptotic upper bound: f(n) = O(g(n)) if there exist positive constants c and n0 such that 0 <= f(n) <= c*g(n) for all n >= n0.
Omega notation represents an asymptotic lower bound, providing a guaranteed minimum growth rate.
Theta notation represents an asymptotically tight bound, holding when an algorithm matches both its upper and lower bounds.
Analyzing growth rates allows engineers to compare algorithmic efficiency independently of hardware differences.

1.2 Recurrence Relations and Divide and Conquer

Divide and conquer algorithms partition a problem into smaller independent subproblems, solve each subproblem recursively, and combine their solutions.
Merge Sort divides an array of size n into two halves, sorts each half recursively, and merges the sorted halves in linear time.
The governing recurrence relation is T(n) = 2T(n/2) + O(n).
Base case formulation occurs when n <= 1, where sorting takes constant O(1) operations.
Solving this recurrence yields an optimal worst-case time complexity of O(n log n).

Figure 1.1: Recurrence Tree for Divide and Conquer Merge Sort

Level 0: T(n) Cost: cn

Level 1: T(n/2) T(n/2) Cost: cn

Level 2: T(n/4) T(n/4) T(n/4) T(n/4) Cost: cn

Total Tree Height: log2(n) | Overall Sum: O(n log n)

1.3 Master Theorem and Recurrence Trees

The Master Theorem provides a cookbook solution for recurrences of the form T(n) = aT(n/b) + f(n), where a >= 1 and b > 1.
Case 1 applies when f(n) is polynomially smaller than n^(log_b a), yielding T(n) = Theta(n^(log_b a)).
Case 2 applies when f(n) and n^(log_b a) are asymptotically equal, yielding T(n) = Theta(n^(log_b a) * log n).
Case 3 applies when f(n) is polynomially larger, yielding T(n) = Theta(f(n)).

1.4 Dynamic Programming Foundations

Dynamic Programming is an algorithmic technique for solving optimization problems by breaking them down into overlapping subproblems.
Optimal substructure means that an optimal solution to the problem contains within its structure optimal solutions to subproblems.
This property is an indispensable requirement for both dynamic programming and greedy algorithms.
Overlapping subproblems means that a recursive formulation visits the same subproblems repeatedly rather than generating new subproblems.
Memoization maintains a top-down lookup table so each subproblem state is calculated at most once.
Tabulation is a bottom-up iterative approach, systematically populating an array from the base cases upward to avoid recursion overhead.

1.5 The 0/1 Knapsack Problem

The 0/1 Knapsack problem considers n items, each with weight w_i and value v_i, and a knapsack with maximum weight capacity W.
The state definition is dp[i][w], representing the maximum value achievable considering a subset of the first i items with capacity at most w.
The state transition equation is:
dp[i][w] = max(dp[i-1][w], dp[i-1][w - w_i] + v_i) when w_i <= w.
If the item exceeds available capacity, we inherit dp[i][w] = dp[i-1][w].
The running time complexity is O(n * W), which is pseudo-polynomial because W is represented in log(W) input bits.

1.6 Graph Algorithms and Shortest Paths

Graph algorithms operate on collections of vertices V and edges E.
Breadth-First Search (BFS) explores all neighboring vertices at the current depth before advancing, finding shortest unweighted paths.
Depth-First Search (DFS) systematically progresses along each branch before backtracking, establishing discovery and finish timestamps.
Topological Sort produces a linear ordering of vertices in a Directed Acyclic Graph (DAG) such that for every directed edge u -> v, u appears before v.
Dijkstra Algorithm computes the single-source shortest path in graphs with non-negative edge weights using a priority queue.
Bellman-Ford Algorithm computes shortest paths in graphs with arbitrary edge weights and detects negative weight cycles in O(V * E) time.